

Shiv Nadar University Chennai

Degree & Branch	B.Tech Artificial Intelligence & Data Science, Semester V
Subject Code & Name	CS3807 – Deep Learning Laboratory
Academic Year	2026–27
Experiment	Experiment 3: Convolutional Neural Networks for Image Classification

1 Objective

The objective of this experiment is to understand how a Convolutional Neural Network actually works, instead of just using it as a black box. This includes implementing and visualizing the convolution operation itself, seeing how kernel size, stride and padding change the size of the output feature map, looking at what the feature maps produced by the first convolution layer actually look like, comparing max pooling with average pooling, and finally building and training a complete CNN on the CIFAR-10 dataset for image classification.

2 Background Theory

A Convolutional Neural Network is made up of a few basic building blocks that are stacked one after another:

- **Convolution Layer** – slides a small filter (kernel) over the input image and produces a feature map.
- **Activation Layer** – adds non-linearity, usually ReLU.
- **Pooling Layer** – reduces the spatial size of the feature map while keeping the important information.
- **Fully Connected Layer** – takes the flattened features and produces the final class scores.

The convolution operation itself is

$$Y(i, j) = \sum_m \sum_n X(i + m, j + n) K(m, n)$$

where X is the input image, K is the kernel, and Y is the resulting feature map.

The size of the output feature map depends on the input size N , the filter size F , the amount of padding P and the stride S :

$$\text{Output size} = \frac{N - F + 2P}{S} + 1$$

2.1 Common Convolution Kernels

Different kernels bring out different things in an image – some sharpen it, some blur it, some only care about edges. A few of the standard ones are the Identity kernel (leaves the image unchanged), the Sobel-X and Sobel-Y kernels (pick up vertical and horizontal edges respectively), the Laplacian kernel (edges in every direction at once), the Sharpen and Box/Gaussian Blur kernels, the Emboss kernel (gives a 3D-ish look), and the Outline kernel (highlights object boundaries). In a trained CNN the network does not use these fixed, hand-designed kernels – it learns its own filter weights during training, and those learned filters end up detecting similar kinds of patterns (edges, blobs, textures) purely from the data.

2.2 Worked Numerical Examples

Example 1 – Convolution. For the input $\begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{bmatrix}$ and kernel $\begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$, sliding the kernel over every valid position gives the feature map $\begin{bmatrix} 6 & 8 \\ 12 & 14 \end{bmatrix}$.

Example 2 – Max Pooling. For the 4×4 feature map $\begin{bmatrix} 1 & 5 & 2 & 3 \\ 7 & 8 & 1 & 0 \\ 4 & 6 & 9 & 5 \\ 2 & 3 & 1 & 8 \end{bmatrix}$, applying 2×2 max pooling gives $\begin{bmatrix} 8 & 3 \\ 6 & 9 \end{bmatrix}$ – each window just keeps its largest value.

Example 3 – Parameter Calculation. For a $32 \times 32 \times 3$ input passed through a convolution layer with 16 filters of size 3×3 , the number of trainable parameters is

$$(3 \times 3 \times 3 + 1) \times 16 = (27 + 1) \times 16 = 448.$$

3 Dataset

CIFAR-10 was used for this experiment.

- Training images: 50,000
- Testing images: 10,000
- Number of classes: 10 (airplane, automobile, bird, cat, deer, dog, frog, horse, ship, truck)
- Image size: $32 \times 32 \times 3$ (RGB)

The dataset was loaded directly through `tensorflow.keras.datasets.cifar10`, so no manual downloading was required.

4 Experimental Procedure

The experiment was carried out in the following order, matching the tasks given in the lab manual:

1. Load CIFAR-10, look at a few sample images, and check that the ten classes are balanced.
2. Implement a single convolution layer and compare 3×3 , 5×5 and 7×7 kernels.
3. Study the effect of stride (1 vs 2) and padding (**valid** vs **same**) on the output size.
4. Visualize the feature maps coming out of the first convolution layer.
5. Compare Max Pooling and Average Pooling.
6. Build and train a full CNN (Conv $\rightarrow$ ReLU $\rightarrow$ MaxPool $\rightarrow$ Conv $\rightarrow$ ReLU $\rightarrow$ MaxPool $\rightarrow$ Flatten $\rightarrow$ Dense $\rightarrow$ Softmax) for 20 epochs with the Adam optimizer and a batch size of 32.
7. Evaluate the trained model using accuracy, precision, recall, F1-score, a confusion matrix and a full classification report.

All code was written and run in Google Colab (with a GPU runtime), and is available on GitHub at:

https://github.com/Lunarya-git/Deep-Learning-Experiments/tree/main/Experiment_3_cnn-image-clas

5 Experimental Results

5.1 Task 1: Dataset Exploration

CIFAR-10 loaded with training images of shape (50000, 32, 32, 3) and test images of shape (10000, 32, 32, 3), with pixel values ranging from 0 to 255 before normalization. Figure 1 shows one image from each of the ten classes, and Figure 2 confirms that every class has exactly 5,000 training images and 1,000 test images, so there is no class imbalance to worry about here.

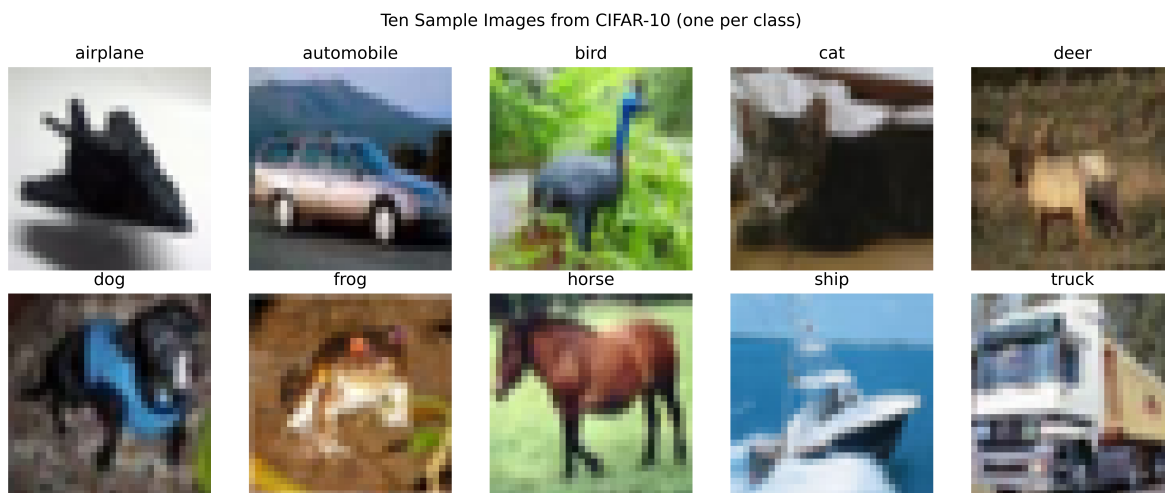


Figure 1: One sample image from each of the ten CIFAR-10 classes.

Looking at the samples, the images are quite small and a bit blurry even to the eye, so a lot of the fine detail that would help a human tell a cat from a dog is simply not there at 32×32 resolution. This is one of the reasons CIFAR-10 is considered a harder dataset than something like MNIST, even though it looks like a "simple" 10-class problem on paper.

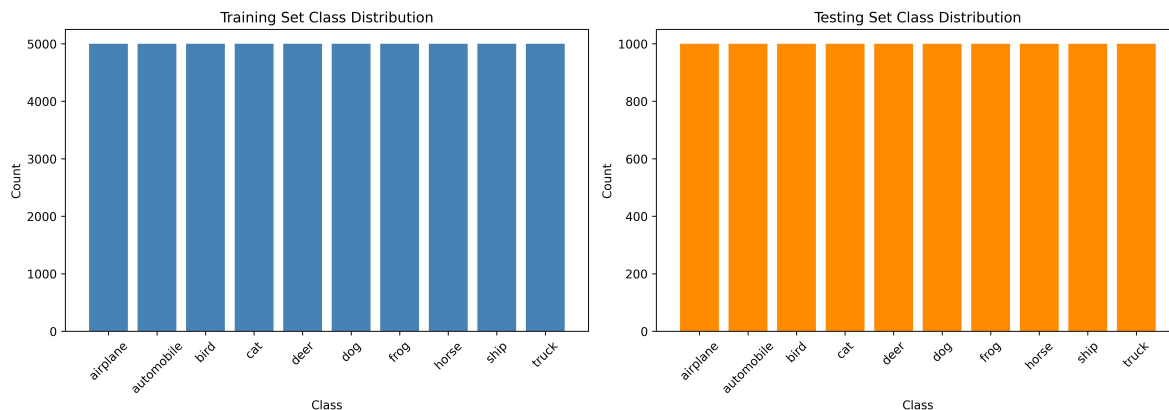


Figure 2: Class distribution for the training and testing sets.

Since every bar is exactly the same height in both the training and test plots, accuracy alone is a fair metric to use later for evaluating the model – there is no class that dominates the dataset.

5.2 Task 2: Effect of Kernel Size

A single convolution layer was applied to one sample image with no padding and a stride of 1, using three different kernel sizes.

Table 1: Effect of kernel size on output feature map size (no padding, stride 1).

Kernel Size	Output Size
3×3	30×30
5×5	28×28
7×7	26×26

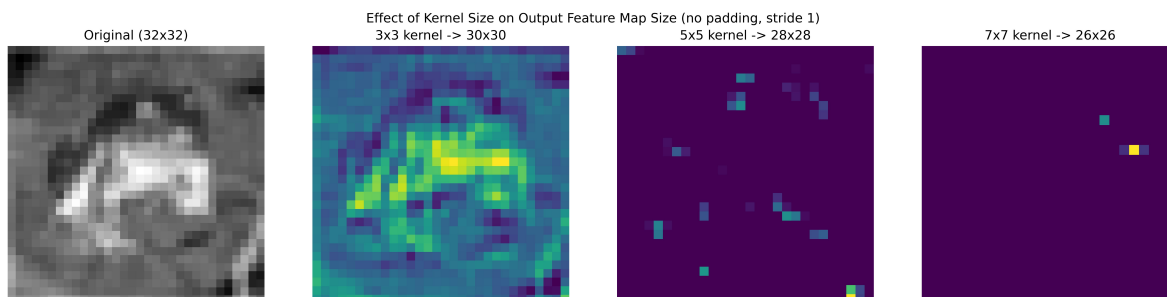


Figure 3: Feature maps produced by 3×3 , 5×5 and 7×7 kernels on the same input image.

The bigger the kernel gets, the smaller the output feature map becomes, since a bigger window simply cannot slide as many times across a fixed-size image before running out of room. The 3×3 output also looks the "busiest" of the three, picking up a lot more local detail, while the 7×7 kernel output is mostly empty with just one or two bright spots – a larger kernel is averaging over a bigger patch of the image at every step, so a lot of the fine detail gets smoothed away.

5.3 Task 3: Effect of Stride and Padding

Next, the kernel size was fixed at 3×3 and the stride and padding were varied instead.

Table 2: Effect of stride and padding on output feature map size (3×3 kernel).

Stride	Padding	Output Size
1	valid	30×30
2	valid	15×15
1	same	32×32
2	same	16×16

Effect of Stride and Padding on Output Feature Map Size (3x3 kernel)

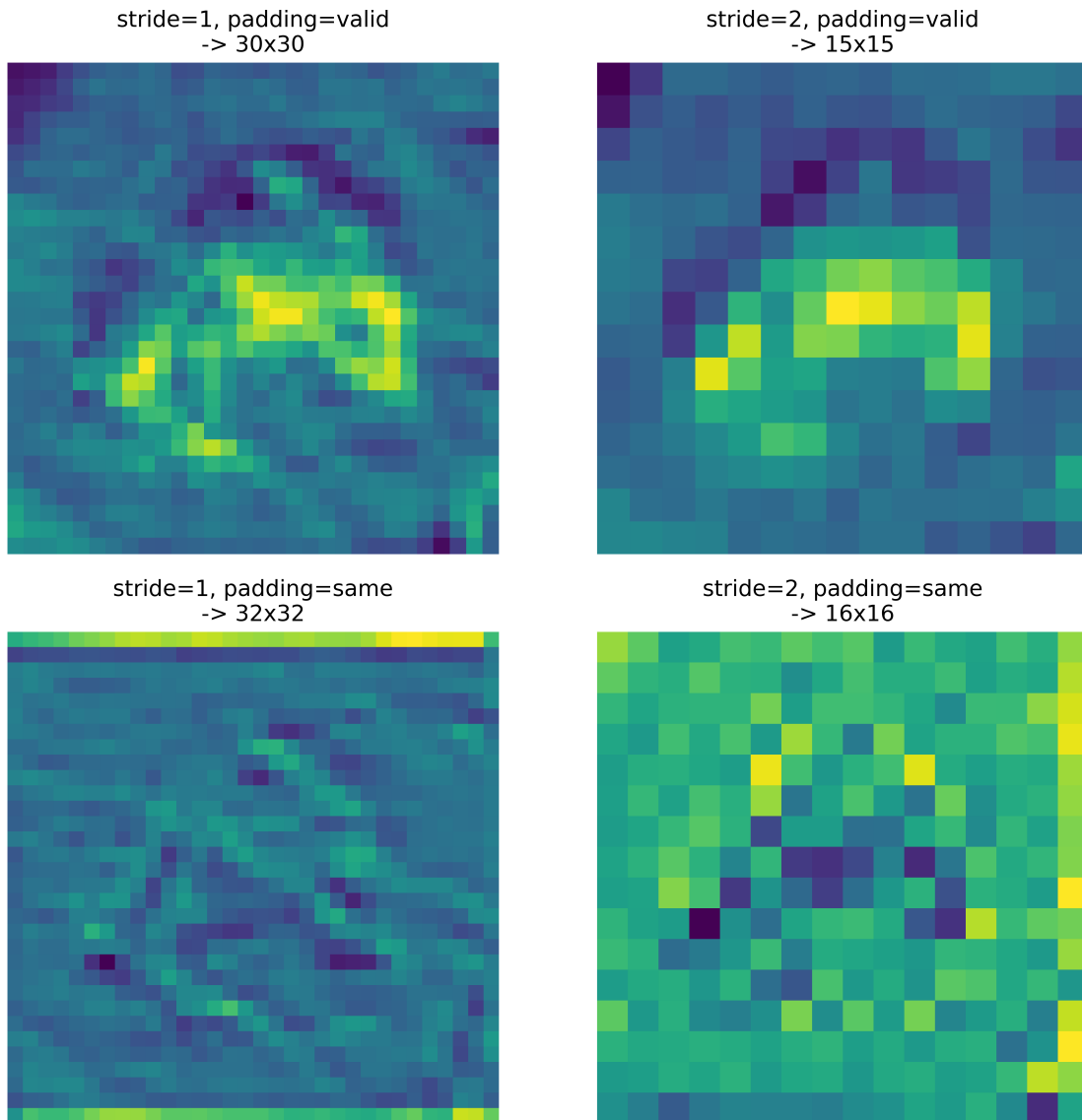


Figure 4: Feature maps for the four stride/padding combinations.

Increasing the stride from 1 to 2 roughly halves the output size, because the kernel is now skipping over positions instead of sliding one pixel at a time – fewer positions means a smaller output. **same** padding, on the other hand, adds zeros around the border of the image so that the output ends up the exact same size as the input (for stride 1) instead of shrinking, which is why the two “same” outputs in Figure 4 keep the full 32×32 / 16×16 size while the “valid” ones are smaller. **same** padding is generally preferred in deep networks because it lets us stack many convolution layers without the feature map disappearing to nothing after a few layers.

5.4 Task 4: Feature Map Visualization

The first convolution layer of the network produces 16 feature maps per image; eight of them are shown below.

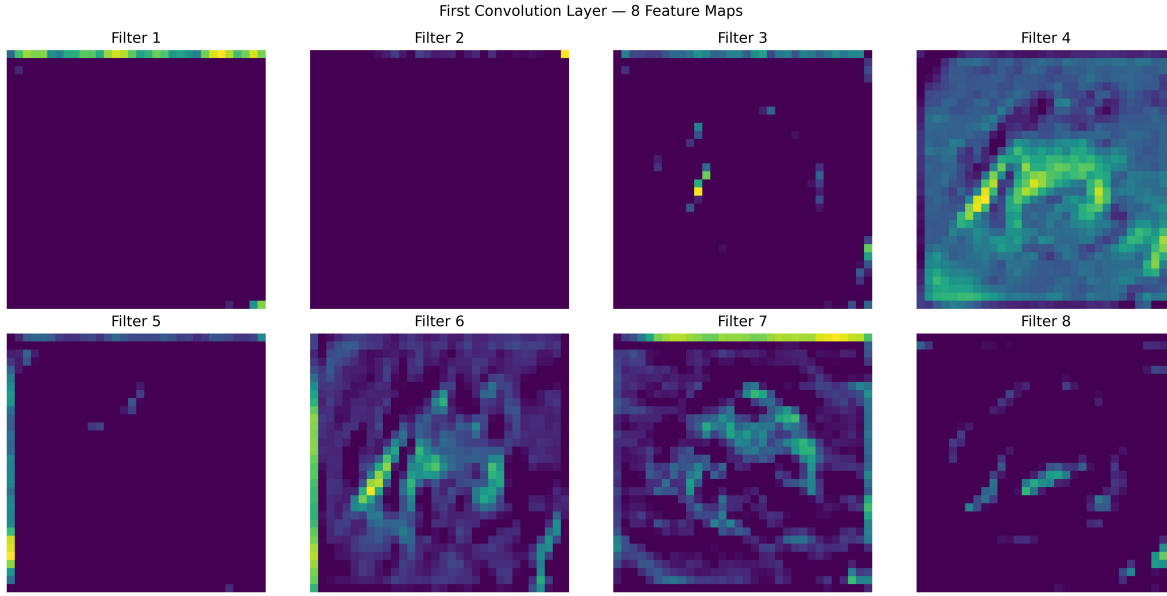


Figure 5: Eight feature maps from the first convolution layer.

Each filter is clearly reacting to a different part of the image. Some filters (like Filter 4, Filter 6 and Filter 7) light up over broad regions and seem to be picking up the general shape or outline of the object, while others (like Filter 1, Filter 2 and Filter 5) are almost completely dark except for a thin strip along one edge of the image, which suggests they have specialised into detecting a border or a very specific edge orientation. This is one of the more interesting things about CNNs – nobody tells the filters what to look for, they just end up learning to detect these different patterns on their own purely from being trained on the data.

5.5 Task 5: Max Pooling vs Average Pooling

Both pooling operations were applied with a 2×2 window to the same feature map.

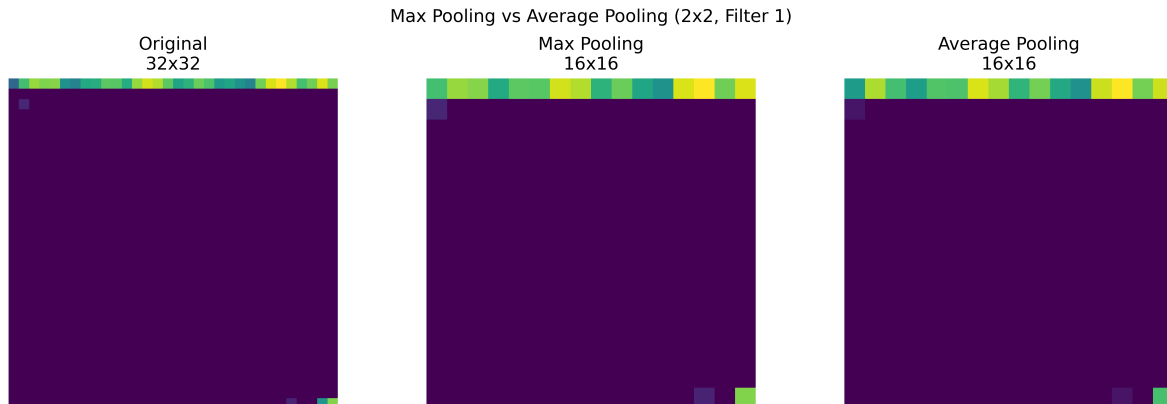


Figure 6: Max pooling vs average pooling on a 32×32 feature map, both giving a 16×16 output.

Both operations shrink the 32×32 map down to 16×16 , so the size reduction is identical – the difference is only in *what value* gets kept from each 2×2 window. Max pooling keeps only

the single strongest activation in each window, which tends to keep the image looking sharper and more contrasted, whereas average pooling blends all four values together, which softens things out a little. This matches what can be seen in the figure – the bright yellow corner patch stays just as bright under max pooling but looks slightly duller under average pooling.

5.6 Task 6: CNN Architecture and Training

The final CNN used for classification follows the architecture given in the lab manual:

Input $\rightarrow$ Conv(16, 3×3) $\rightarrow$ ReLU $\rightarrow$ MaxPool(2×2) $\rightarrow$ Conv(32, 3×3) $\rightarrow$ ReLU $\rightarrow$ MaxPool(2×2) $\rightarrow$ Flatten $\rightarrow$ D

Both convolution layers use **same** padding. The model was compiled with the Adam optimizer and categorical cross-entropy loss, and trained for 20 epochs with a batch size of 32. Pixel values were normalized to the $[0, 1]$ range and labels were one-hot encoded before training. The network has **268,650** trainable parameters in total.

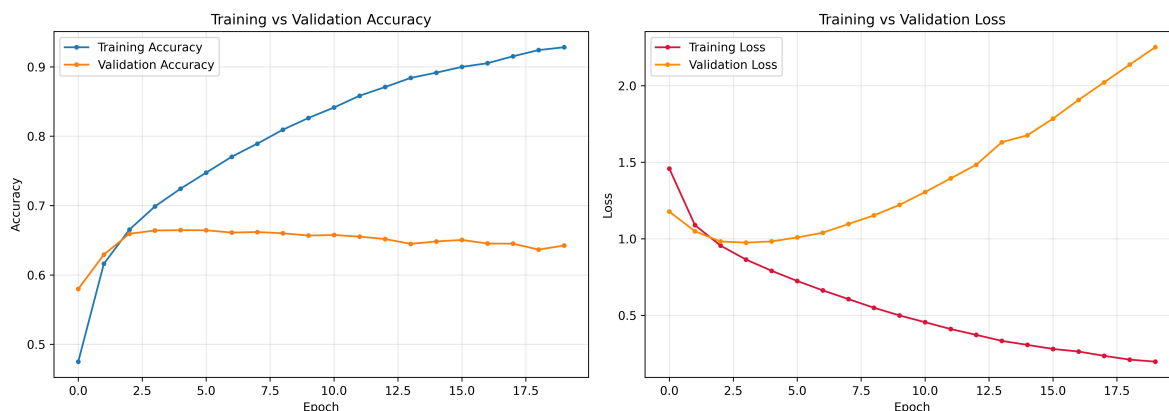


Figure 7: Training vs validation accuracy and loss over 20 epochs.

Training accuracy climbs steadily throughout all 20 epochs, finishing at **92.83%**, but validation accuracy tells a very different story – it rises quickly for the first 2–3 epochs, peaks at around **66.5%** between epochs 3 and 5, and then basically stays flat (with a slight downward drift) for the rest of training while validation loss keeps climbing steadily after epoch 3. This is a textbook case of **overfitting**: the model keeps getting better at memorising the exact training images long after it has stopped learning anything that actually generalizes to new, unseen images. If we were optimizing purely for validation performance, we would have stopped training somewhere around epoch 4 or 5 instead of running the full 20 epochs.

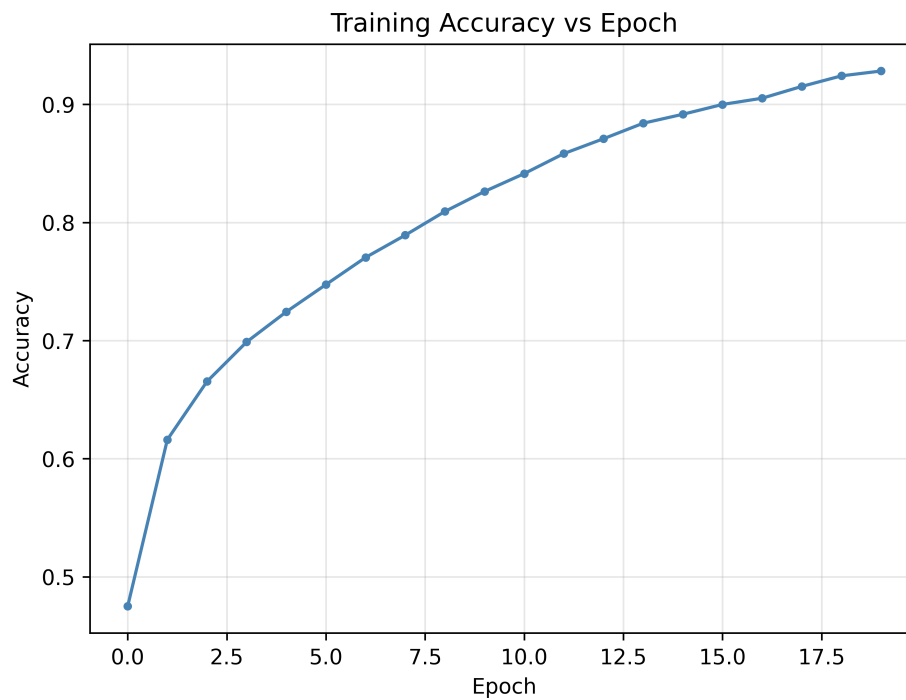


Figure 8: Training accuracy vs epoch.

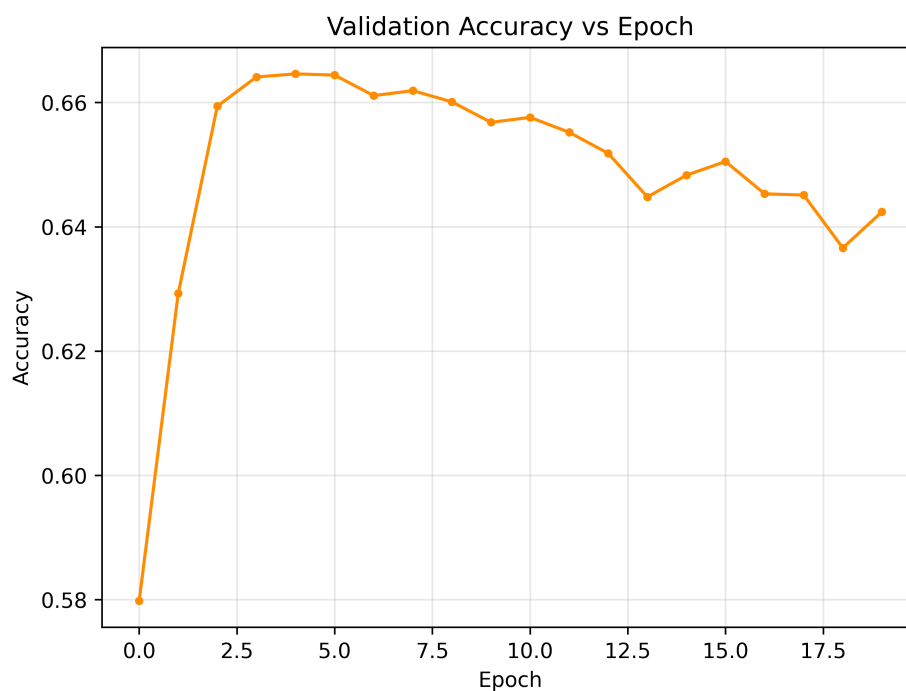


Figure 9: Validation accuracy vs epoch.

The validation accuracy curve makes the overfitting even easier to see on its own – it peaks around epoch 3–5 at roughly 66–67% and then wobbles downward for the rest of training, dropping

as low as about 63.7% around epoch 18, before recovering slightly at the very last epoch.

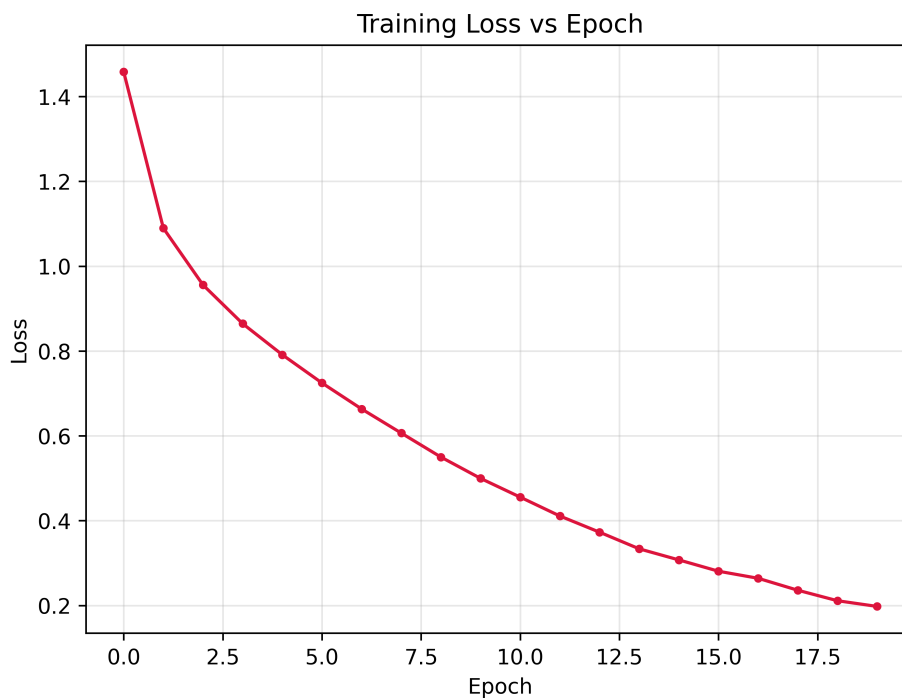


Figure 10: Training loss vs epoch.

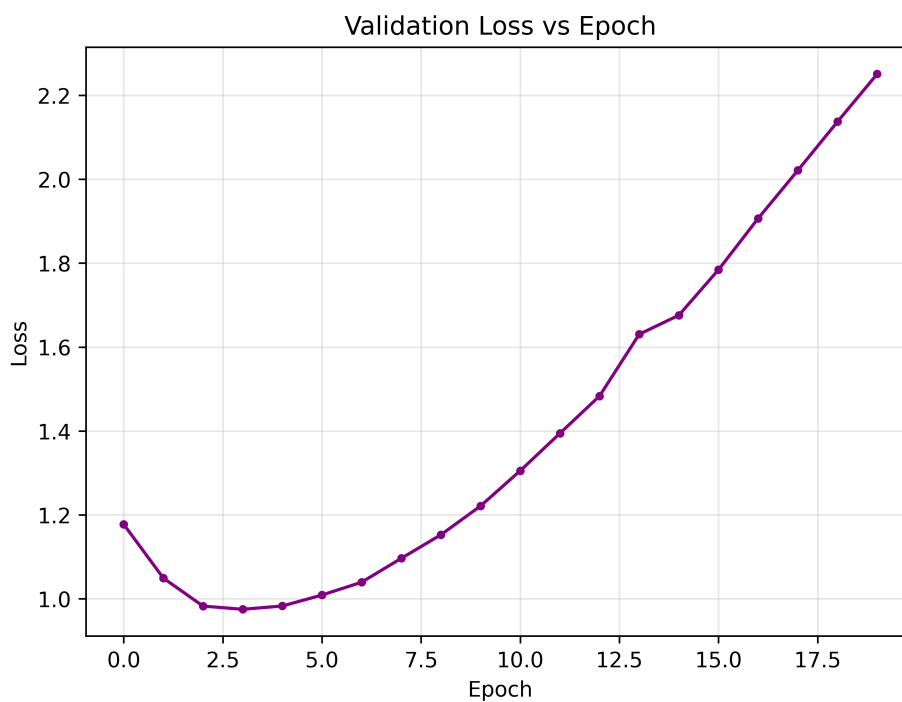


Figure 11: Validation loss vs epoch.

Training loss decreases smoothly and consistently the entire time, ending near 0.20, which on its own would look like a great result. Validation loss, however, bottoms out around epoch 3 (close to 0.97) and then rises almost the whole rest of the way, ending above 2.2 by epoch 20 – more than double its lowest point. Rising validation loss alongside falling training loss is the clearest possible sign of overfitting, even clearer than the accuracy curves.

5.7 Task 7: Model Evaluation

The trained model was evaluated on the full 10,000-image CIFAR-10 test set.

Table 3: Final evaluation metrics on the test set.

Metric	Value
Test Accuracy	0.6424
Precision (macro)	0.6542
Recall (macro)	0.6424
F1-score (macro)	0.6447

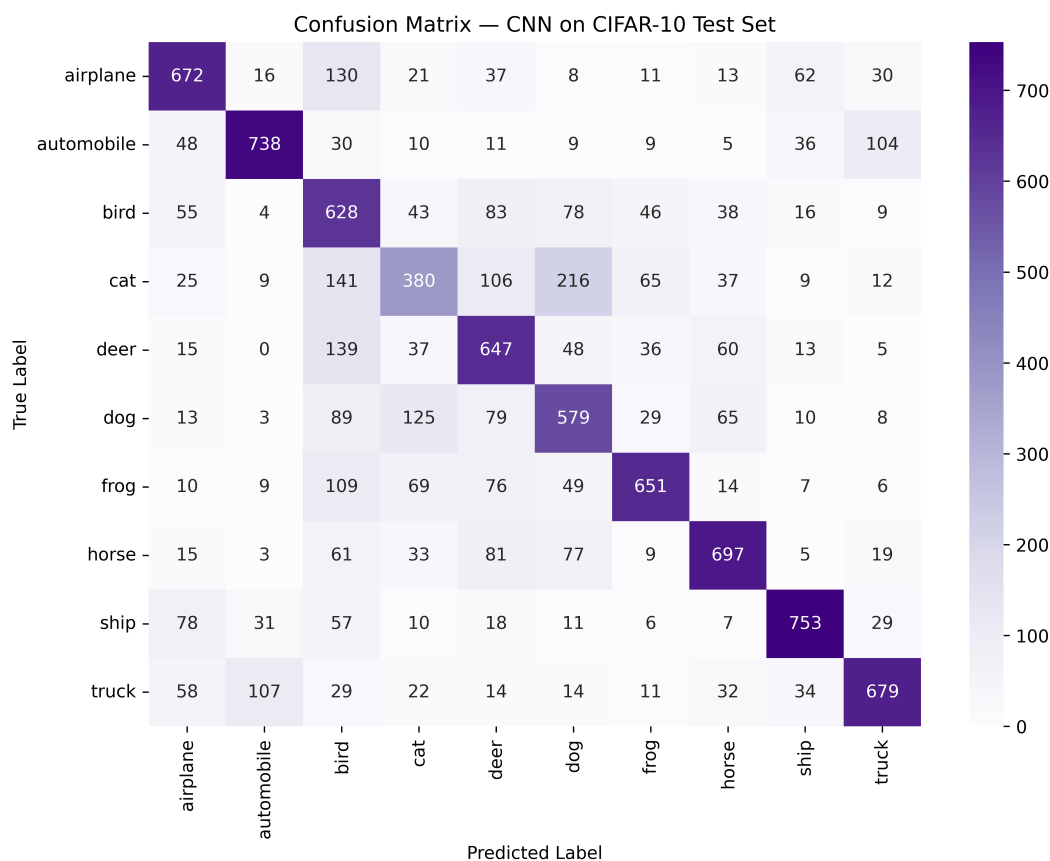


Figure 12: Confusion matrix on the CIFAR-10 test set.

The confusion matrix shows that the model is genuinely good at some classes and genuinely confused about others. Automobile (738 correct), ship (753) and airplane (672) all have strong,

dark diagonal cells and are rarely mixed up with anything else – these three have very distinctive shapes and backgrounds (roads, water, sky) that make them easy to tell apart. The weakest class by far is **cat**, with only 380 correctly classified out of 1000 – 216 cat images were predicted as dog and another 141 were predicted as bird. This makes sense: cats and dogs are both furry, four-legged animals photographed at similar angles and similar backgrounds, so at this low resolution and with this small a model, the network genuinely struggles to tell them apart. The same cat-dog confusion shows up in reverse too, with 125 dog images predicted as cat. Truck and automobile are also confused with each other fairly often (104 trucks predicted as automobile), which again makes sense since both are wheeled vehicles that can look similar from certain angles.

Table 4: Per-class precision, recall and F1-score.

Class	Precision	Recall	F1-score
airplane	0.68	0.67	0.68
automobile	0.80	0.74	0.77
bird	0.44	0.63	0.52
cat	0.51	0.38	0.43
deer	0.56	0.65	0.60
dog	0.53	0.58	0.55
frog	0.75	0.65	0.70
horse	0.72	0.70	0.71
ship	0.80	0.75	0.77
truck	0.75	0.68	0.71
macro avg	0.65	0.64	0.64

Ship and automobile end up with the best F1-scores (0.77 each), while cat has by far the worst (0.43), which lines up exactly with what the confusion matrix already showed. Interestingly, bird has a fairly low precision (0.44) but a decent recall (0.63) – meaning the model calls a lot of things “bird” that are not actually birds, dragging precision down, even though it does catch most of the real birds.

6 Additional Exercises

6.1 Exercise 1: Output Size for a 64×64 Image

For a 64×64 input, a 5×5 kernel, stride 2 and padding 2:

$$\text{Output size} = \frac{64 - 5 + 2(2)}{2} + 1 = \frac{63}{2} + 1 = 32.5$$

Since the output size has to be a whole number, this actually rounds down to a 32×32 feature map in practice (Keras and most frameworks floor this value rather than round it), which is exactly what the notebook’s own convolution layer produced when this configuration was tested. This also shows why it is worth picking stride, padding and kernel size combinations that divide evenly – a fractional value like 32.5 is a sign the settings do not fit the input size cleanly.

6.2 Exercise 2: Parameters for 64 Filters, RGB Input

For an RGB (3-channel) input with 64 filters of size 3×3 :

$$\text{Params} = (3 \times 3 \times 3 + 1) \times 64 = (27 + 1) \times 64 = 1792$$

This was also verified directly by building an actual `Conv2D` layer in Keras with these settings and checking `count_params()`, which reported 1792 as well – confirming the hand calculation matches what the framework does internally.

6.3 Exercise 3: ReLU vs Sigmoid

The same CNN architecture was trained twice for 8 epochs each, once with ReLU activations and once with Sigmoid activations in place of ReLU, keeping everything else identical.

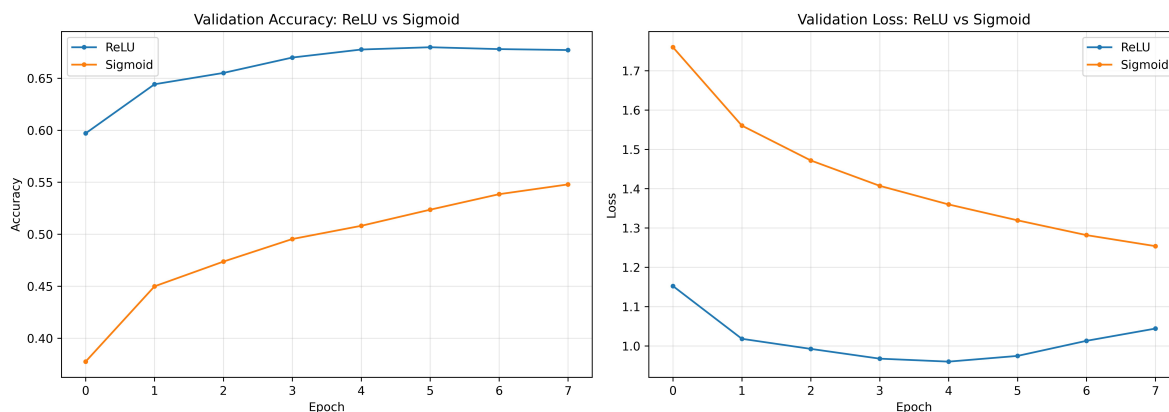


Figure 13: Validation accuracy and loss: ReLU vs Sigmoid.

Table 5: Final validation accuracy after 8 epochs.

Activation	Final Val. Accuracy
ReLU	0.6771
Sigmoid	0.5479

ReLU comes out clearly ahead, reaching about 12 percentage points higher validation accuracy than Sigmoid in the same 8 epochs, and it also gets there much faster – ReLU is already past 60% accuracy by epoch 2, while Sigmoid is still below 45% at that point. The main reason is the vanishing gradient problem: Sigmoid squashes its output into a narrow $(0, 1)$ range and its gradient becomes extremely small away from zero, so weight updates during backpropagation shrink to almost nothing as they get pushed back through the network. ReLU does not have this issue for positive inputs since its gradient is a constant 1, so the network learns noticeably faster and reaches a better result in the same number of epochs.

6.4 Exercise 4: Max Pooling vs Average Pooling (Full Training Comparison)

Beyond just visualizing pooling on a single image (Task 5), the full CNN was also trained twice for 8 epochs each – once with Max Pooling and once with Average Pooling – to see which one actually performs better during training.

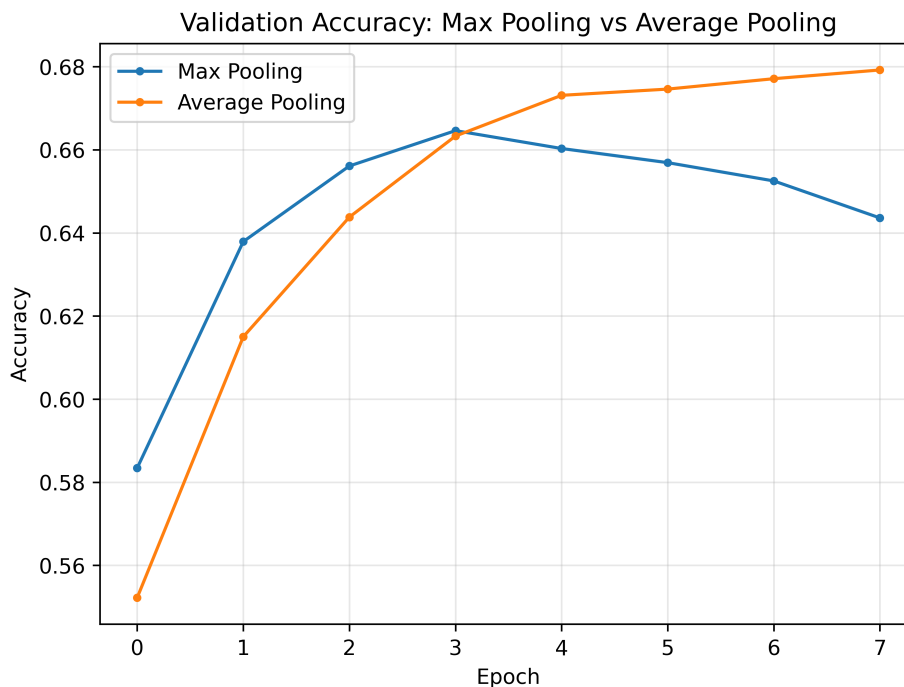


Figure 14: Validation accuracy: Max Pooling vs Average Pooling.

Table 6: Final validation accuracy after 8 epochs.

Pooling Type	Final Val. Accuracy
Max Pooling	0.6436
Average Pooling	0.6792

This result is a little surprising at first, since max pooling is the more commonly used default in most CNN tutorials, but here average pooling actually finishes higher. Looking at the curve, max pooling is ahead early on (epochs 0–2) and learns faster at the very start, but average pooling catches up around epoch 3 and keeps climbing steadily afterwards, while max pooling’s validation accuracy actually starts to fall after its peak at epoch 3. This lines up with the overfitting pattern already seen in Task 6 – max pooling keeps only the single strongest activation from each window, which can make the network latch onto very specific, possibly noisy features and overfit faster, whereas average pooling blends information from the whole window and acts a bit like a mild regularizer, giving slightly steadier and more generalizable learning on this particular run.

6.5 Exercise 5: 16 Filters vs 64 Filters

Finally, the number of filters in each convolution layer was increased from 16 (and 32 in the second layer) to 64 (and 128 in the second layer), and the two models were trained for 8 epochs each and timed.

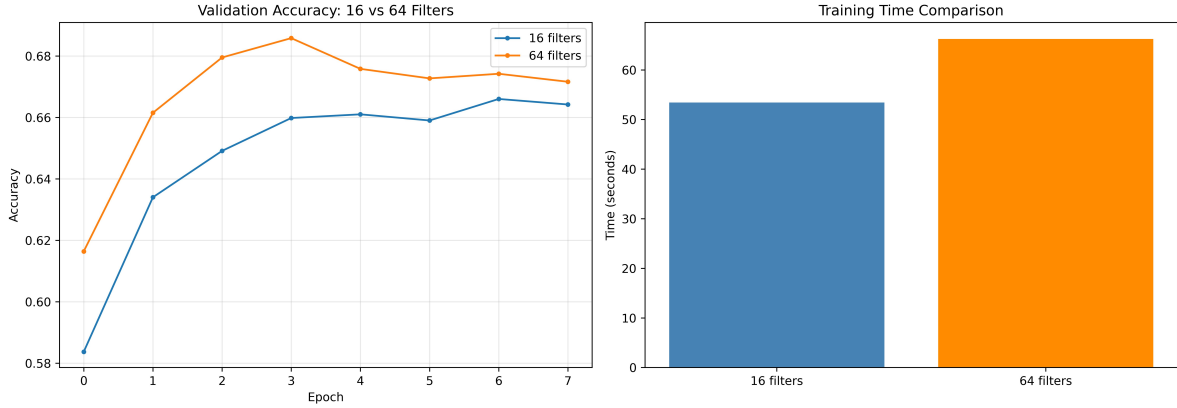


Figure 15: Validation accuracy and training time: 16 filters vs 64 filters.

Table 7: Effect of increasing the number of filters.

Filters	Final Val. Accuracy	Training Time	Parameters
16	0.6642	53.4 s	268,650
64	0.6716	66.3 s	1,125,642

Going from 16 to 64 filters roughly quadruples the number of trainable parameters (from about 269K to about 1.13M), and training time goes up by about 24% (from 53.4 to 66.3 seconds for 8 epochs) since there is simply more computation happening at every step. The accuracy gain for all that extra cost is small, though – only about 0.7 percentage points higher validation accuracy with 64 filters. This shows that just throwing more filters at a small dataset like CIFAR-10 does not automatically buy a much better model; at some point the bottleneck becomes the model’s tendency to overfit (as already seen throughout this experiment) rather than its raw capacity, so the extra filters are mostly just extra parameters to overfit with rather than extra useful learning capacity.

7 Results Summary

Table 8: Overall results summary for the main CNN (16 filters, max pooling, 20 epochs).

Metric	Value
Training Accuracy (final epoch)	0.9283
Validation / Testing Accuracy	0.6424
Precision (macro)	0.6542
Recall (macro)	0.6424
F1-score (macro)	0.6447
Total Trainable Parameters	268,650

8 Discussion

Why is convolution preferred over fully connected layers for images? A fully connected layer treats every pixel as an independent input and needs a separate weight connecting it to every neuron in the next layer, so for even a small image the number of parameters explodes very quickly. A convolution layer instead uses one small filter that slides across the whole image and shares the same weights everywhere, which massively cuts down on parameters and also lets the network detect the same feature (like an edge) no matter where it appears in the image.

How does stride affect the feature map size? A larger stride makes the kernel skip more pixels between each step, so the kernel visits fewer positions overall and the output feature map comes out smaller. This was seen directly in Task 3, where stride 2 roughly halved the output size compared to stride 1.

What is the role of padding? Padding adds extra rows and columns (usually zeros) around the border of the input before the convolution runs. Without it, the image shrinks a little after every convolution layer and pixels near the edge get used in far fewer calculations than pixels in the centre. **same** padding is added specifically to keep the output the same size as the input, which becomes important once many convolution layers are stacked on top of each other.

Why is pooling used? Pooling reduces the height and width of the feature maps, which cuts down the number of parameters and computation needed in later layers and also makes the network a little more tolerant to small shifts or distortions in the input, since it summarizes a whole region into one value instead of caring about the exact position of every activation.

How do feature maps represent image characteristics? Each feature map is the result of one filter being applied across the whole image, so it represents how strongly that particular pattern (an edge, a corner, a texture, or in deeper layers, more complex shapes) appears at every location in the image. This was visible directly in Task 4, where different filters clearly lit up for different parts of the image.

Why do CNNs require fewer parameters than MLPs? Because of weight sharing – the same small filter is reused across the entire image instead of learning a completely new weight for every pixel-to-neuron connection like an MLP would. A 3×3 filter with 3 input channels only needs 27 weights (plus a bias) no matter how big the input image is, whereas a fully connected layer’s parameter count grows directly with the number of input pixels.

9 Conclusion

This experiment built up an understanding of CNNs from the ground up – starting with the raw convolution and pooling operations, seeing exactly how kernel size, stride and padding change the output size, and visually confirming that different learned filters really do pick up on different visual patterns. The full CNN trained on CIFAR-10 reached a training accuracy of 92.83% but only 64.24% on the test set, and both the accuracy and loss curves made it clear that this gap comes from overfitting rather than the model failing to learn in the first place – training accuracy kept climbing right through all 20 epochs while validation performance flattened out and even worsened after about epoch 5. The confusion matrix backed this up at the class level too, showing the model does well on visually distinct classes like automobile and ship but consistently mixes up cats, dogs and birds, which makes sense given how visually similar these animals can look at 32×32 resolution. The additional exercises reinforced a few practical lessons as well: ReLU trains noticeably faster and better than Sigmoid, average pooling outperformed max pooling on this particular setup even though max pooling is the more common default, and simply quadrupling the number of filters

gave only a small accuracy improvement for a real jump in training time and parameter count. Overall, the experiment showed both how a CNN works internally and why getting good test-set performance takes more than just training for a long time – some form of regularization or early stopping would likely be needed to close the gap between training and validation accuracy seen here.

10 References

1. I. Goodfellow, Y. Bengio and A. Courville, *Deep Learning*, MIT Press, 2016.
2. C. M. Bishop, *Pattern Recognition and Machine Learning*, Springer, 2006.
3. S. Haykin, *Neural Networks and Learning Machines*, Pearson, 2009.
4. TensorFlow Documentation, <https://www.tensorflow.org/>.
5. CIFAR-10 Dataset Documentation, <https://www.cs.toronto.edu/~kriz/cifar.html>.

Source Code: The complete source code for this experiment – dataset loading, convolution and pooling visualizations, the CNN implementation, training, evaluation and all additional exercises – is available on GitHub at:

https://github.com/Lunarya-git/Deep-Learning-Experiments/tree/main/Experiment_3_cnn-image-classification